

Bi-Weekly Research Progress Report

Project: IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent: Sarthak Mushrif
Reporting Period: Weeks 9 & 10
Hours Logged: 20 Hours

1. Executive Summary of Progress

During the fifth reporting period, the focus was centered on ensuring system fault tolerance and finalizing the Two-Phase Commit (2PC) protocol. The primary achievement was the successful integration of AWS DynamoDB as a high-speed Write-Ahead Log (WAL). This integration ensures that the distributed state of any transaction is persisted safely outside of the proxy's volatile memory. With the WAL in place, Phase 2 of the 2PC protocol was completed, culminating in the first successful end-to-end distributed UPDATE transaction spanning multiple database shards.

2. Detailed Task Breakdown (20 Hours)

- **AWS DynamoDB WAL Integration (8 Hours):** Configured the AWS SDK v2 within the project. Designed the DynamoDB table schema (TransactionID as Partition Key, with State and Timestamp attributes) to act as the Write-Ahead Log. Implemented the logging service to record PREPARED, COMMITTED, and ABORTED states.
- **Two-Phase Commit - Phase 2 Execution (6 Hours):** Developed the decision logic for the Coordinator. If all shards return a positive vote during Phase 1, the system now synchronously writes a COMMITTED record to DynamoDB *before* asynchronously dispatching the final COMMIT command to the physical PostgreSQL shards via the HikariCP pool.
- **End-to-End Distributed UPDATE Testing (4 Hours):** Orchestrated an integration test simulating a cross-shard transaction (e.g., deducting balance from Shard A and adding to Shard B). Verified that the ANTLR4 compiler correctly parsed the intent, the router successfully identified the two target shards, and the 2PC engine executed the locks and commits sequentially.
- **Failure & Rollback Simulation (2 Hours):** Authored tests to intentionally fail one shard during Phase 1. Verified that the Coordinator correctly logged an ABORTED state to DynamoDB and successfully rolled back the locked rows on the surviving shard.

3. Challenges & Roadblocks

- **DynamoDB I/O Latency:** Initial synchronous writes to AWS DynamoDB introduced a ~15ms blocking delay in the main transaction thread, reducing overall throughput. This was resolved by migrating to the DynamoDbAsyncClient, allowing the proxy to leverage non-blocking I/O and chain the WAL logging process with the database commit process using Java CompletableFuture.
- **Phantom Locks on Proxy Crash:** Discovered a vulnerability during testing where, if the IsoShard Proxy process is forcefully terminated *after* Phase 1 but *before* Phase 2, the underlying RDS instances retain their row locks indefinitely. This necessitates the development of a dedicated Crash Recovery daemon in the next sprint.

4. Objectives for Next Reporting Period (Weeks 11 & 12)

- Implement the **Crash Recovery / Reconciliation Daemon**, which will read unresolved transactions from DynamoDB on proxy startup and issue compensating rollbacks or commits to clear phantom locks.
- Containerize the IsoShard Proxy application using **Docker** to standardize the runtime environment and prepare for deployment onto AWS EC2 Auto-Scaling groups.
- Develop "Predicate Pushdown" optimization logic in the AST Visitor for SELECT queries to prevent full-cluster table scans.

Appendix A: Technical Artifact - DynamoDB WAL Logger

The following Java class demonstrates the asynchronous Write-Ahead Log integration using the AWS SDK v2. This guarantees that transaction decisions are safely persisted before communicating with the database shards.

```
1 import software.amazon.awssdk.services.dynamodb.DynamoDbAsyncClient;
2 import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
3 import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
4 import java.util.Map;
5 import java.util.concurrent.CompletableFuture;
6
7 public class DynamoDbWalLogger {
8
9     private final DynamoDbAsyncClient dynamoClient;
10    private final String tableName = "IsoShard_WAL";
11
12    public DynamoDbWalLogger() {
13        // Initializes using default AWS credentials provider chain
14        this.dynamoClient = DynamoDbAsyncClient.create();
15    }
16
17    /**
18     * Asynchronously logs the transaction state to AWS DynamoDB.
19     * This MUST complete before Phase 2 dispatch to ensure crash
20     * recoverability.
21     */
22    public CompletableFuture<Void> logTransactionState(String txId, String
23    state) {
24        Map<String, AttributeValue> item = Map.of(
25            "TransactionId", AttributeValue.builder().s(txId).build(),
26            "State", AttributeValue.builder().s(state).build(),
27            "Timestamp", AttributeValue.builder().n(String.valueOf(System.
28    currentTimeMillis())).build()
29        );
30
31        PutItemRequest request = PutItemRequest.builder()
32            .tableName(tableName)
33            .item(item)
34            .build();
35
36        // Return the CompletableFuture so the Coordinator can chain the next
37        // action
38        return dynamoClient.putItem(request).thenApply(response -> null);
39    }
40 }
```

Listing 1: DynamoDbWalLogger.java